

Vacation-Rental Listing — React + Java Architecture

Current repository architecture for the guest-facing listing page. Read top → bottom as request flow.

Repository baseline: the React page currently reads the typed listing model from `frontend/src/data/listing.ts` and serves images/fonts from `public/assets`. The Java API is a separate, dependency-free REST boundary exposing health and listing metadata; the production path below shows how this boundary scales into the full marketplace.

CLIENTS

Web (React + Vite)

Desktop and mobile browser client. Static Vite build with the listing UI, overlays, keyboard behavior, and local assets.

iOS / Android

Native apps consuming the same GraphQL/REST edge APIs.

Partners / SEO bots

Prerendered HTML + structured data for crawlers and channel partners.

EDGE & DELIVERY

Static asset delivery

Vite serves the built frontend and shared public assets. A CDN can be placed in front of the static output for production delivery.

Image CDN

On-the-fly resize/format (AVIF/WebP) & responsive `srcset` from object storage. Signed URLs.

WAF + DDoS + Bot mgmt

Rate limiting, bot detection (the checkpoint we bypassed for capture), edge auth token checks.

HTTP API boundary

Current Java endpoints are available at `localhost:8080/api`. In production, the same REST boundary sits behind auth, rate limits, tracing, and an API gateway.

APPLICATION SERVICES · STATELESS, AUTOSCALED ON KUBERNETES

Java listing API

Current dependency-free server exposes health and listing metadata. Scale it as a stateless service with replicas, caching, validation, and an OpenAPI contract.

Search & Discovery

Geo + availability + ranking. Backed by a search cluster; personalization via ML features.

Availability & Pricing

Calendar, dynamic pricing, promotions. High write throughput, eventually consistent to search.

Booking & Payments

Reservations, holds, escrow, payouts via PSP. Strong consistency, sagas, idempotency keys.

Reviews & Messaging

Guest ↔ host chat, reviews/ratings. Fan-out via event bus; moderation pipeline.

Cross-cutting: Auth/Identity service, Notification service (email/push/SMS), Media-ingest & moderation, Fraud/Trust & Safety.

DATA STORES

Primary DB — PostgreSQL

Production source of truth for users, listings, availability, bookings, and reviews. Read replicas; partition by region or host as traffic grows.

Search cluster — OpenSearch/Elasticsearch

Denormalized listing + geo + availability index. Rebuilt from change events.

Cache — Redis

Hot listings, sessions, rate-limit counters, availability lookups.

Object storage — S3/GCS

Original photos & media, fronted by the image CDN.

ASYNC & ANALYTICS

Event bus — Kafka

Domain events (`booking.created`, `listing.updated`) drive search re-indexing, notifications, and the data lake — services stay decoupled.

Stream + batch — Flink / Spark

Real-time availability sync, fraud signals, pricing features.

Data lake + warehouse

S3 + BigQuery/Snowflake for BI, experimentation, ML training.

ML platform

Ranking, dynamic pricing, recommendations; features served back to Search.

PLATFORM & DELIVERY

Deployment

Build the React app with Vite and run the Java API separately. The current local setup uses ports 5173 and 8080.

CI/CD

GitHub Actions → build/test → container registry → ArgoCD/GitOps. IaC via Terraform.

Observability

OpenTelemetry traces, Prometheus/Grafana metrics, centralized logs, SLOs & alerting.

Resilience

Multi-region active-passive, automated failover, PITR backups, autoscaling on RPS/queue depth.

☐ Clients ☐ Edge & delivery ☐ Application services ☐ Data & async ☐ Platform

Solid implementation today: React/Vite client, local typed listing data, public media assets, and Java health/listing endpoints. Production evolution: move listing persistence behind the Java service and add the stores, queues, security, and delivery layers shown above.